

Grundlagen der Cloudtechnologie

Cloudtechnologie

Zusammenfassung und Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 7. Juni 2026

Inhaltsverzeichnis

1	Grundbegriffe	3
1.1	Cloud	3
1.2	Compute	3
1.3	Storage	3
1.4	Region	3
1.5	Verfügbarkeitszonen	4
1.6	CapEx – Capital Expenditure	4
1.7	OpEx – Operational Expenditure	4
1.8	Skalierung – Scalability	4
1.8.1	Horizontale Skalierung	4
1.8.2	Vertikale Skalierung	4
1.9	Hochverfügbarkeit – Availability	4
1.9.1	SLA – Service Level Agreement	4
1.10	Elastizität – Elasticity	4
2	Cloud Begriffe	5
2.1	Cloud consumer	5
2.2	Cloud Provider	5
2.3	Clouddirector	5
2.4	Cloud broker	5
2.5	Cloud carrier	5
3	Cloud nach NIST	6
3.1	NIST – National Institute of Standards and Technology	6
3.2	On-Demand Self-Service	6
3.3	Broad Network Access	6
3.4	Resource Pooling	6
3.5	Rapid Elasticity	6
3.6	Measured Service	6
4	Virtualisierung	7
4.1	Wie nutzt eine Cloud Virtualisierung ?	7
4.1.1	Virtuelle Maschinen (Virtual Machine)	7
4.1.2	Virtuelles Netzwerk	7
4.1.3	Speichervirtualisierung	7
4.2	Wie funktioniert Virtualisierung	8
4.2.1	Hypervisor	8
5	Cloud Liefermodelle	9
6	Servicemodelle	10
6.1	Infrastructure as a Service (IaaS)	10
6.2	Platform as a Service (PaaS)	10
6.3	Software as a Service (SaaS)	11
6.4	Shared Responsibility Model	12
6.4.1	Sicherheitsaufgaben von Kunde und Provider	12
6.4.2	Warum verändern sich die Verantwortlichkeiten?	12
7	Cloud Services	13
7.1	Compute	13

7.2	Serveless	13
8	Rechenzentrum	15
8.1	Komponenten eines Rechenzentrum	15
8.2	Aufrecht erhaltung eine Rechenzentrum	16
9	Internet Service Provider – ISP	17
10	Hyperscaler	18
11	Docker und Kubernetes	20
11.1	Docker	20
11.1.1	Image	20
11.1.2	Container	20
11.1.3	Container Regsitrie	20
11.2	Kubernetes	21
11.2.1	Pods	21
11.2.2	Node	21
11.2.3	Cluster	21
11.2.4	Instanzen	21
11.3	Kubenrets Funktion	22
12	Cloud Architektur	23
12.1	Cloud Native	23
12.1.1	DevOps	24
12.1.2	CI/CD – Continuous Integration / Continuous Delivery	24
12.1.3	Microservices	25
12.2	Architektur-Grundlagen	25
12.2.1	Warum wichtig?	25
12.2.2	Technische Schulden	25
12.2.3	Ziele einer Lösungs-Architektur	25
12.2.4	Architektur-Perspektiven	26
12.2.5	Architektur-Frameworks: Governance	27
12.2.6	Architektur-Frameworks: Praxisorientiert	27
12.3	C4	29

1 Grundbegriffe

1.1 Cloud

Eine Cloud beschreibt das Outsourcen von Rechenaufgaben und Speicher an einem Computer der nicht der lokale Computer ist. Die Cloud kann unterschiedliche Aufgaben übernehmen, meistens sind es Compute und Storage.

1.2 Compute

Compute in Cloud Kontext bedeutet das eine Cloud Rechenleistung bereitstellt. Dies kann vielseitig geschehen von einfachen VMs, über Managed Services zum Hosten eigener Anwendungen bis zu SaaS Produkten wie Microsoft 365.

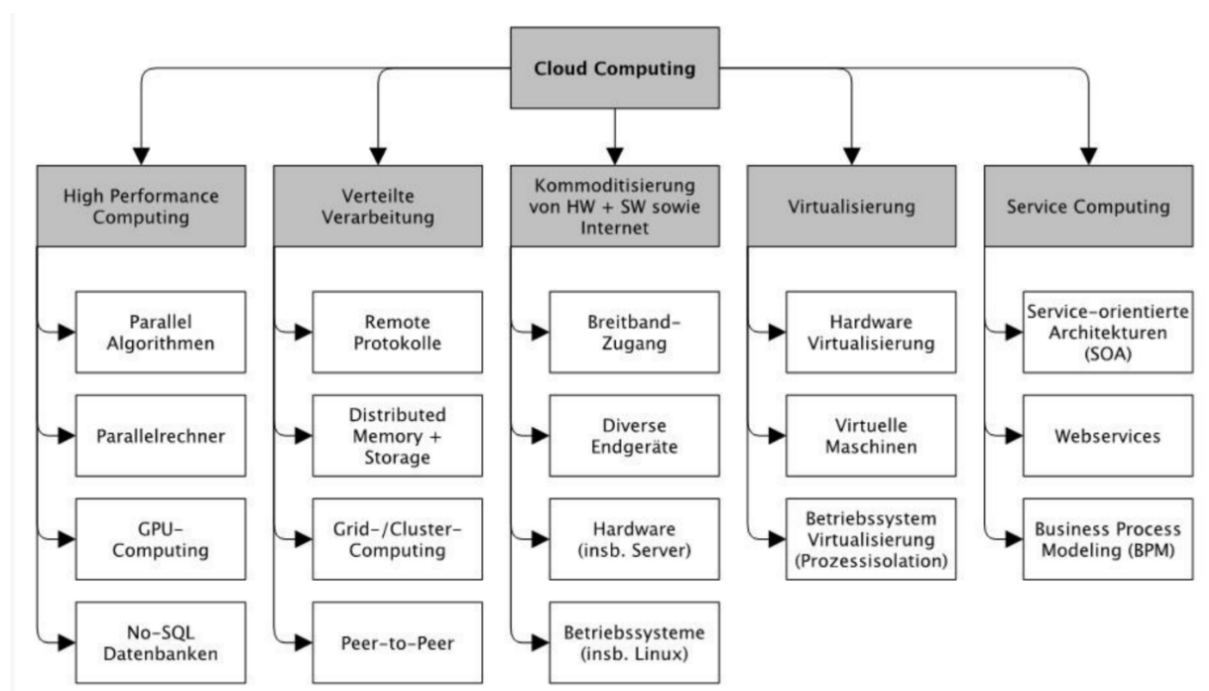


Abbildung 1. Cloud Computing erklärt

1.3 Storage

Storage in Cloud Kontext bedeutet die Bereitstellung von Speicher. Dies kann geschehen über die verschiedensten Wege zu einem einfachen Cloud Speicher ähnlich zu einer Festplatte, Speicher für Webseiten um Nutzerdaten zu speichern aber auch Speicher integriert in SaaS Produkten wie Microsoft 365 aber auch beispielsweise Netflix.

1.4 Region

Eine Cloud-Region ist ein geografischer Bereich, der aus mehreren Verfügbarkeitszonen besteht. Regionen sind meist nach Himmelsrichtung oder Lage benannt, zum Beispiel *Deutschland West Central* oder *North Europe* bei Azure. Die Wahl der Region bestimmt, wo Daten physisch gespeichert werden (Datenresidenz) und beeinflusst die Latenz zu den Endnutzern. Jede Region ist physisch und logisch von anderen Regionen isoliert, sodass ein Ausfall einer Region andere nicht beeinträchtigt.

1.5 Verfügbarkeitszonen

Eine Verfügbarkeitszone (engl. *Availability Zone*) ist eine physisch getrennte Einheit innerhalb einer Region. Sie besteht aus einem oder mehreren Rechenzentren mit eigener Stromversorgung, Kühlung und Netzwerkanbindung. Verfügbarkeitszonen innerhalb einer Region sind über redundante Hochgeschwindigkeitsverbindungen mit geringer Latenz verbunden. Durch die Verteilung von Diensten auf mehrere Zonen können Anwendungen auch bei einem Zonenausfall verfügbar bleiben (Hochverfügbarkeit).

1.6 CapEx – Capital Expenditure

CapEx oder auch Kapitalausgaben sind eine einmalige Investition. Im Cloud Kontext sind das oft der Kauf von Servern. Ein CapEx Ausgabe ist über mehrere Jahre abschreibbar. Das Problem bei einem einmaligen Serverkauf sind die hohen Kosten und das ein Server eine begrenzte Zeit hat in welcher er performant ist. Man kann ab einem gewissen Zeitpunkt nur noch schwierig Hardware nachrüsten.

1.7 OpEx – Operational Expenditure

OpEx sind laufende Kosten so wie Cloud Abonnements. OpEx wird direkt verbucht.

1.8 Skalierung – Scalability

Skalierung im Cloud Kontext bedeutet Rechenleistung, Speicher und Netzwerkbandbreite dynamisch an sich ändernde Workloads anzupassen. Es wird in zwei Arten unterteilt.

1.8.1 Horizontale Skalierung

Bei horizontaler Skalierung wird ein Workload auf mehreren Servern ausgeführt bzw. ausgeweitet.

1.8.2 Vertikale Skalierung

Ein bestehender Server wird aufgerüstet durch beispielsweise mehr RAM.

1.9 Hochverfügbarkeit – Availability

Hochverfügbarkeit bedeutet das eine in der Cloud gehostete Anwendung immer erreichbar ist. Wie viele Ausfälle im Jahr prozentual passieren dürfen ist in einem SLA (Service Level Agreement) festgehalten.

1.9.1 SLA – Service Level Agreement

In einem SLA ist Uptime, meist als Nines also z.B. 5 Nines → 99,999, Reaktion und Lösungszeit, Leistungsparameter, Ausstiegsstrategien und Datenportabilität sowie Sicherheit und Compliance geregelt.

1.10 Elastizität – Elasticity

Elastizität im Cloud Kontext bedeutet das eine Anwendung automatisch skaliert werden kann.

2 Cloud Begriffe

2.1 Cloud consumer

Ein Cloud Consumer ist der Nutzer von durch eine Cloud bereitgestellten Diensten. Dieser entscheidet über seine eigenen Abonnements. Ein Beispiel ist wenn man sich für ein Claude – Also den KI Chatbot – Abo entscheidet ist man ein Cloud Consumer. Dasselbe gilt für iCloud, Spotify, Apple Music und Netflix. Im Enterprise Kontext ist das noch mal anders dort sind typische Abonnements Microsoft 365, Salesforce oder auch Abonnements bei Cloud Providern wie bei Azure.

2.2 Cloud Provider

Ein Cloud Provider stellt die Infrastruktur und die Services einer Cloud bereit. Abseits davon sind die Provider für Betrieb und Sicherheit der Dienste verantwortlich. Beispiele sind die Hyperscaler wie Azure, AWS und Google aber auch europäische Anbieter wie beispielsweise Hetzner sind Cloud Provider.

2.3 Clouddauditor

Ein Cloud Auditor prüft Rechenzentren von Cloud Providern und ob diese sich an den Datenschutz und Compliance halten.

2.4 Cloud broker

Ein Cloud Broker ist ein unabhängiger Anbieter der zwischen Cloud Consumer und Provider kommuniziert. Ein Beispiel wäre ein Systemhaus das Lizenzen einer Cloud kauft und Consumer diese samt Support und Konfiguration weiter verkauft.

2.5 Cloud carrier

Ein Cloud Carrier ist für eine sichere und stabile Datenübertragung von Cloud Consumer zu Cloud Provider verantwortlich.

3 Cloud nach NIST

Die NIST definiert Cloud nicht über Marketingbegriffe, sondern über 5 technische und organisatorische Eigenschaften.

3.1 NIST – National Institute of Standards and Technology

NIST ist eine US-amerikanische Behörde. Die NIST ist gleichzeitig eine Großforschungseinrichtung mit den Forschungsbereichen Messtechnik von physikalisch-technischen Konstanten und heutzutage auch Informationssicherheit.

3.2 On-Demand Self-Service

Ein Cloud Nutzer kann eigenständig Services erwerben. Ohne IT-Abteilung, Callcenter oder Vermittler. Auf einer Cloud sollte man jederzeit zugreifen können sowie die Cloud jederzeit kündigen können. Zusätzlich läuft die Cloud über automatisierte Schnittstellen.

3.3 Broad Network Access

Eine Cloud sollte von überall mit internetfähigem Gerät erreichbar sein. Da der Cloud-Service flächendeckend verfügbar ist.

3.4 Resource Pooling

Bei einer Cloud werden Ressourcen geteilt. Das bedeutet mehrere Nutzer teilen sich die Rechenleistung eines Servers. Um Datenschutz der jeweiligen Region oder Zone zu gewährleisten muss der Cloud Provider entsprechende Maßnahmen treffen.

3.5 Rapid Elasticity

Elastizität im Cloud Kontext bedeutet das eine Anwendung automatisch skaliert werden kann, also das ein richtiges Ressourcenlevel aus Rechenleistung, Speicher, Bandbreite und Speicher variabel gewählt wird. Dies ist wichtig um möglichst gut auf Nutzung reagieren zu können ohne das ein Ausfall passiert. Ein beliebtes Beispiel ist hier der Online Shop an Black Friday.

3.6 Measured Service

Ein Service in der Cloud sollte messbar sein. Das ist gerade wichtig für Pay-as-you-go, also das man nur das zahlt was man auch wirklich nutzt. Dadurch ist eine höhere Transparenz möglich, wodurch man Kosten besser kalkulieren kann.

4 Virtualisierung

Virtualisierung ist ein schon lange bestehendes Konzept. Bei Virtualisierung wird auf einem physischen Computer durch Software eine andere, isolierte Umgebung abstrahiert. Bekannt ist das Konzept beispielsweise durch Oracle VirtualBox. Allerdings ist Virtualisierung als die Abstraktion von Hardware Ressourcen durch Software auch der Grundstein für Cloudtechnologie. Durch Virtualisierung wird Ressource Pooling überhaupt erst ermöglicht, sonst könnten keine voneinander getrennten Umgebungen auf einem Server existieren.

4.1 Wie nutzt eine Cloud Virtualisierung ?

4.1.1 Virtuelle Maschinen (Virtual Machine)

Bei einer VM, kurz für Virtuelle Maschine, wird eine eigenständige, isolierte Instanz auf einem physischen System abstrahiert. Dadurch kann man beispielsweise auf einem Windows-PC ein Linux-Betriebssystem innerhalb einer VM ausführen. In der Cloud wird dies für Ressource Pooling genutzt, also das Aufteilen eines physischen Servers in mehrere kleinere, unabhängige Instanzen. So werden Infrastructure as a Service (IaaS) und dynamische Skalierung überhaupt erst ermöglicht.

4.1.2 Virtuelles Netzwerk

Durch ein virtuelles Netzwerk wird die Kommunikation zwischen VMs sowie mit externen Systemen überhaupt erst ermöglicht. Durch Peering, also das direkte Verbinden zweier virtueller Netzwerke miteinander, können Services untereinander kommunizieren, ohne den Umweg über das öffentliche Internet nehmen zu müssen. Durch VPNs, kurz für Virtual Private Network, wird zudem eine verschlüsselte Direktverbindung zu On-Premises-Rechenzentren ermöglicht, um sicher mit Cloud-Services zu kommunizieren.

4.1.3 Speichervirtualisierung

Bei der Speichervirtualisierung werden mehrere physische Speichermedien durch Software zu einem einzigen logischen Speicherpool zusammengefasst und abstrahiert. Das ermöglicht beispielsweise Services wie Google Drive, iCloud oder im Enterprise Kontext Azure Managed Disks.

4.2 Wie Funktioniert Virtualisierung

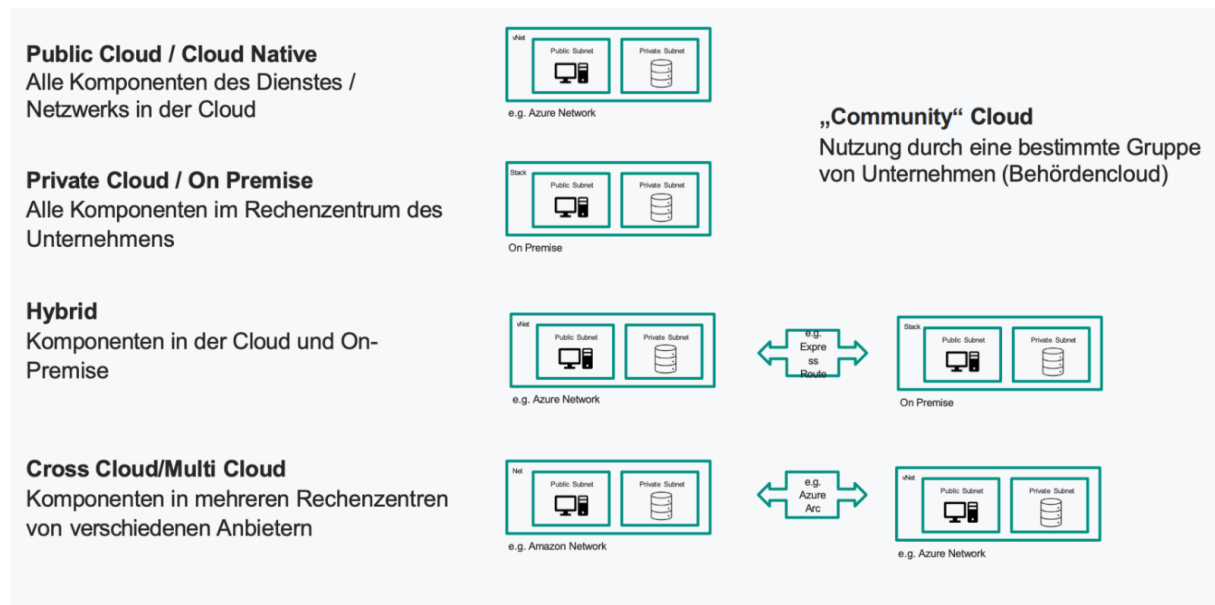
4.2.1 Hypervisor

Ein Hypervisor ist die Software die Arbeitsspeicher, Rechenleistung und Speicher in Pools zusammenfasst. Dadurch können auf eine Physischen Maschine viele VMs erstellt werden. Der Hypervisor stellt den einzelnen virtuellen Maschinen die zugewiesenen Ressourcen zur Verfügung und verwaltet die Planung der VM-Ressourcen im Verhältnis zu den physischen Ressourcen. Die physische Hardware übernimmt nach wie vor die Ausführung, das heißt die CPU führt nach wie vor CPU-Befehle aus, wie sie beispielsweise von den VMs angefordert werden, während der Hypervisor die Planung übernimmt.

Merkmal	Typ 1 (Bare-Metal)	Typ 2 (Gehostet)
Ausführung	Direkt auf der Hardware	Auf einem Host-Betriebssystem
Host-OS	Keins, ersetzt das OS	Benötigt ein Host-OS
Ressourcenzuweisung	Direkt durch den Hypervisor	Über das Host-OS
Beispiele	VMware ESXi, KVM	Oracle VirtualBox, VMware Workstation

Tabelle 1. Vergleich Hypervisor Typ 1 und Typ 2

5 Cloud Liefermodelle



Modell	Kosten	Sicherheit	Customization / Flexibilität	Notwendige Ressourcen
Public Cloud	Kostengünstigste Architektur	Hoher Sicherheitsstandard, keine volle Kontrolle (reicht möglicherweise nicht aus)	Begrenzt, abhängig vom Dienst des Cloud-Service-Providers (CSP)	Kein detailliertes Infrastruktur-Know-how erforderlich
Private Cloud	Teuerste Architektur	Keine garantierte Sicherheit. Alle Sicherheitsstandards können auf eigene Kosten umgesetzt werden	Infrastruktur kann beliebig konfiguriert werden	Detailliertes Know-how für alle erforderlichen Infrastrukturebenen erforderlich
Hybride Cloud	Könnte je nach Bedarf Vorteile haben	Alle Sicherheitsstandards umsetzbar. Die Verbindung zur Cloud muss gesichert sein	Alle Möglichkeiten beider Architekturen	Detailliertes Know-how für alle erforderlichen Infrastrukturebenen erforderlich

Tabelle 2. Vergleich der Cloud-Bereitstellungsmodelle

6 Servicemodelle

6.1 Infrastructure as a Service (IaaS)

Bei IaaS wird dem Nutzer ausschließlich die Infrastruktur bereitgestellt. Ein typisches Beispiel sind virtuelle Maschinen. Der Nutzer ist selbst für das Betriebssystem und alle darauf installierten Anwendungen verantwortlich. Load Balancer und ähnliche Dienste sind nicht vorkonfiguriert und müssen manuell eingerichtet werden.

Flexibilität: Hoch, man ist bei der Nutzung vollständig frei.

Verantwortung: Hoch, nur die physische Hardware muss nicht selbst verwaltet werden.

Typische Anwendungsfälle: Entwicklungs-Workloads und Hosten von Legacy-Systemen.

Public Cloud	Private Cloud	Hybrid Cloud
IaaS in der Public Cloud wird oft genutzt, um virtuelle Server bereitzustellen, insbesondere für Startups. Zusätzlich ist IaaS in der Public Cloud kostengünstig und skalierbar.	Große Unternehmen können IaaS auch in ihrer Private Cloud nutzen, um maximale Kontrolle über Computing-Ressourcen zu haben.	Die meiste Rechenleistung befindet sich in der Private Cloud, nur bei Spitzenlasten wird auf die Public Cloud gewechselt. Auch Cloud Bursting genannt.

6.2 Platform as a Service (PaaS)

Bei PaaS bewegt man sich in Richtung Managed Services. Ein Beispiel wäre eine verwaltete SQL-Datenbank in Azure. Viele Aspekte wie Load Balancer und Skalierung werden bereits vom Anbieter übernommen, was die Entwicklung deutlich vereinfacht.

Flexibilität: Hoch, allerdings mit deutlich weniger Konfigurationsaufwand als bei IaaS.

Verantwortung: Mittel, viele Dienste werden bereits vom Provider verwaltet.

Typische Anwendungsfälle: Hosten von Webanwendungen, verwaltete Datenbanken in der Cloud.

Public Cloud	Private Cloud	Hybrid Cloud
PaaS wird in der Public Cloud genutzt, um Anwendungen von Entwicklern bereitzustellen. Beispielsweise soll eine Webanwendung für das ganze Unternehmen zur Verfügung gestellt werden – dies wird in Azure beispielsweise über einen Azure App Service getan.	PaaS-Plattformen können auch in einem unternehmenseigenen Rechenzentrum eingerichtet werden, sodass Entwicklungsteams in einer sicheren Umgebung testen können.	Entwicklungsteams testen in einer privaten PaaS-Umgebung und deployen die Anwendung dann in eine öffentliche PaaS-Umgebung.

6.3 Software as a Service (SaaS)

Bei SaaS erhält der Nutzer ein fertiges Produkt, das nur noch konfiguriert werden muss. Eigener Code kann nicht eingebunden werden, alles funktioniert über Customizing.

Flexibilität: Gering, alles skaliert automatisch, eigene Eingriffe sind kaum möglich.

Verantwortung: Gering, der Nutzer ist nur für die eigenen Daten verantwortlich.

Typische Anwendungsfälle: Produkte wie Microsoft 365.

Public Cloud	Private Cloud	Hybrid Cloud
SaaS-Produkte wie Microsoft 365, Gmail oder Salesforce sind am weitesten verbreitet. Eine Anwendung läuft komplett in der Public-Cloud-Infrastruktur eines Anbieters. Die Anwendungen werden über das Internet bereitgestellt.	SaaS-Produkte können bei Unternehmen mit hohen Sicherheitsanforderungen in der eigenen Cloud-Infrastruktur gehostet werden. Beispielsweise können auch KI-Modelle wie Claude oder Gemini bei einem Unternehmen selbst gehostet werden.	Dies ist eine häufige Kombination in Unternehmen: Sensible Daten liegen On-Premises in der Private Cloud, beispielsweise auch ein ERP-System. Daneben werden aber auch SaaS-Anwendungen wie Teams oder Gmail genutzt.

6.4 Shared Responsibility Model

Das Shared Responsibility Model zeigt auf, wo die Verantwortlichkeiten bei Cloud-Modellen liegen und wie viel vom Nutzer bzw. Unternehmen selbst gemanagt werden muss im Vergleich zu dem, was der Cloud-Provider übernimmt.

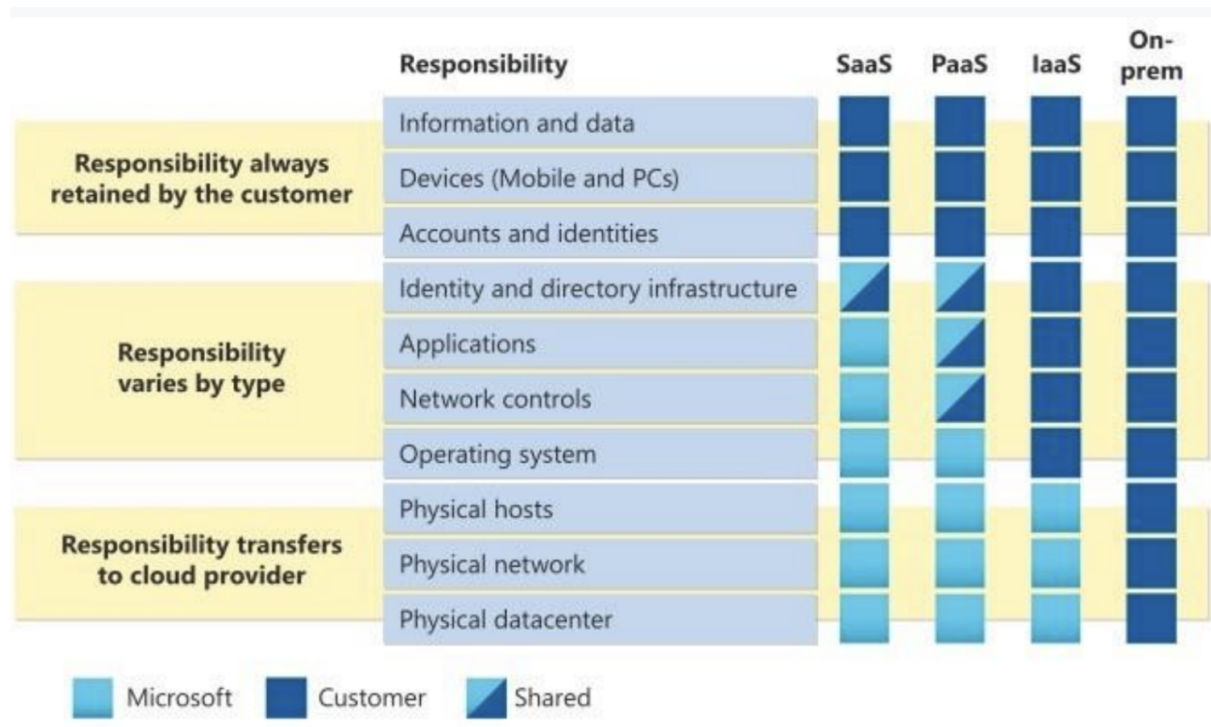


Abbildung 2. Shared Responsibility Model

6.4.1 Sicherheitsaufgaben von Kunde und Provider

Der Kunde ist hauptsächlich für die Sicherheit der Client-Geräte verantwortlich, also beispielsweise dafür, dass Mitarbeiter sichere Passwörter verwenden. Zusätzlich können RBAC und Conditional Access aktiviert werden, sodass Endnutzer automatisch mehr Sicherheit erhalten. Authentifizierung, die beispielsweise SSO ermöglicht, wird vom Cloud-Provider bereitgestellt. Der Provider ist seinerseits vollständig für die Sicherheit im Rechenzentrum und seiner eigenen Services zuständig. Für die Sicherheit der eigenen Daten ist der Kunde verantwortlich, muss dafür aber oft nur wenige Konfigurationen vornehmen, beispielsweise Disaster Recovery durch Zonenredundanz.

6.4.2 Warum verändern sich die Verantwortlichkeiten?

Die verschiedenen Servicemodelle haben unterschiedliche Anwendungsfälle. IaaS wird genutzt, wenn möglichst viel selbst konfiguriert werden soll, beispielsweise für das Hosten von Legacy-Systemen oder Entwicklungs-Workloads. Die Kosten für den Compute werden dabei oft im Voraus bezahlt, beispielsweise über Azure Reservations. PaaS wird für eigene Cloud-Native-Anwendungen genutzt, hat aber grundlegende Dinge bereits vorkonfiguriert, sodass keine VM manuell eingerichtet werden muss. SaaS sind vollständig abgeschlossene Softwareprodukte, die einfach über die Cloud laufen, wie beispielsweise Microsoft 365.

7 Cloud Services

Ein Cloud-Dienst (englisch: Cloud Service) bezeichnet eine Dienstleistung, die von einem Cloud-Provider über das Internet bereitgestellt wird. Dabei stellt der Anbieter Ressourcen wie Compute, Speicher oder Software auf Abruf zur Verfügung, ohne dass der Nutzer eigene physische Infrastruktur betreiben muss.

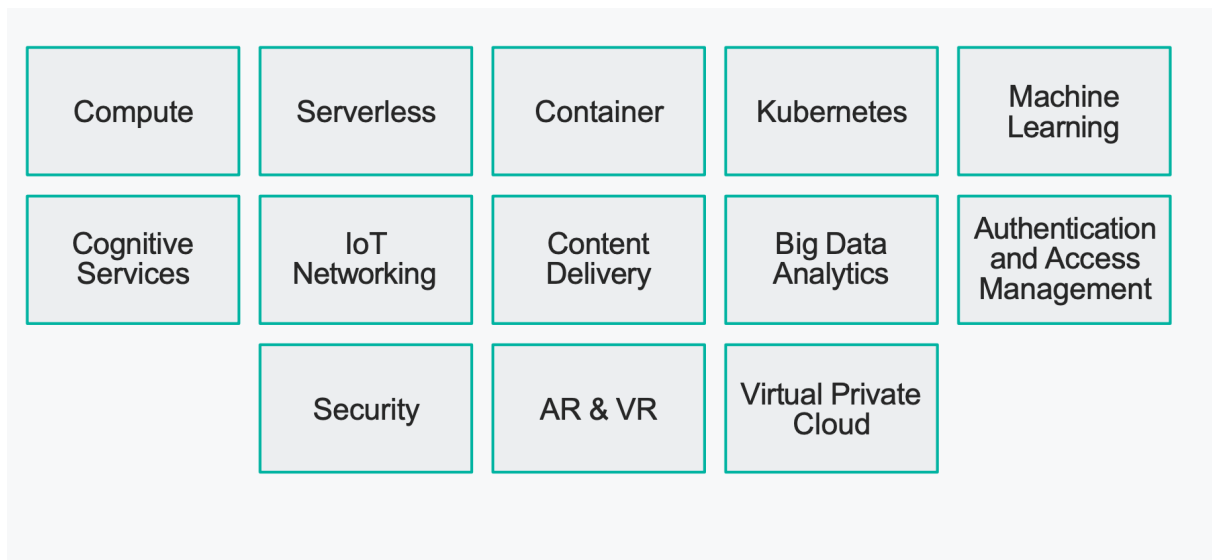


Abbildung 3. Grundlegende Services in Der Cloud

7.1 Compute

Compute bedeutet das bereitstellen von Rechenleistung. Also kann Compute hosten einer Webanwendung bedeutet oder des hosten eine KI modell wie Sonnet oder GPT 5 für ein Unternehmen.

7.2 Serveless

Serveless bedeutet das man Code ohne Pflege eines Server Hosten kann. Es ist oft teil von Ressourcen in Azure z.B. der Managed Service für Postgre Datenbanken. Ein sonder fall ist FaaS. Dies wird Später noch mal detaillert erklärt.

Definiton FaaS – Function as a Service

FaaS baut auf Serveless auf und emrölicht Backend cod Serveless zu hosten, Durch Genreische sprachen wie z.B. Python.

Aspekt	Server Architecture	Serverless Architecture
Infrastructure Management	Entwickler verwalten zugrunde liegende Server oder VMs.	Der Cloud-Anbieter verwaltet die Infrastruktur (Server).
Resource Allocation	Ressourcen, die basierend auf dem erwarteten Workload bereitgestellt werden.	Ressourcen, die dynamisch basierend auf dem Bedarf zugewiesen werden.
Scaling	Die Skalierung umfasst in der Regel manuelle oder automatisierte Prozesse.	Automatische Skalierung basierend auf Workload.
Cost Model	Die Kosten beinhalten Vorabinvestitionen und das laufende Management.	Pay-per-Use-Preismodell basierend auf Funktionsaufrufen.
State Management	Anwendungen behalten den Status auf der Serverseite bei.	Serverlose Funktionen sind von Natur aus zustandslos.

Tabelle 3. Vergleich Server Architecture und Serverless Architecture

8 Rechenzentrum

Moderne Rechenzentren haben viel Anforderung die sie Erfüllen müssen.

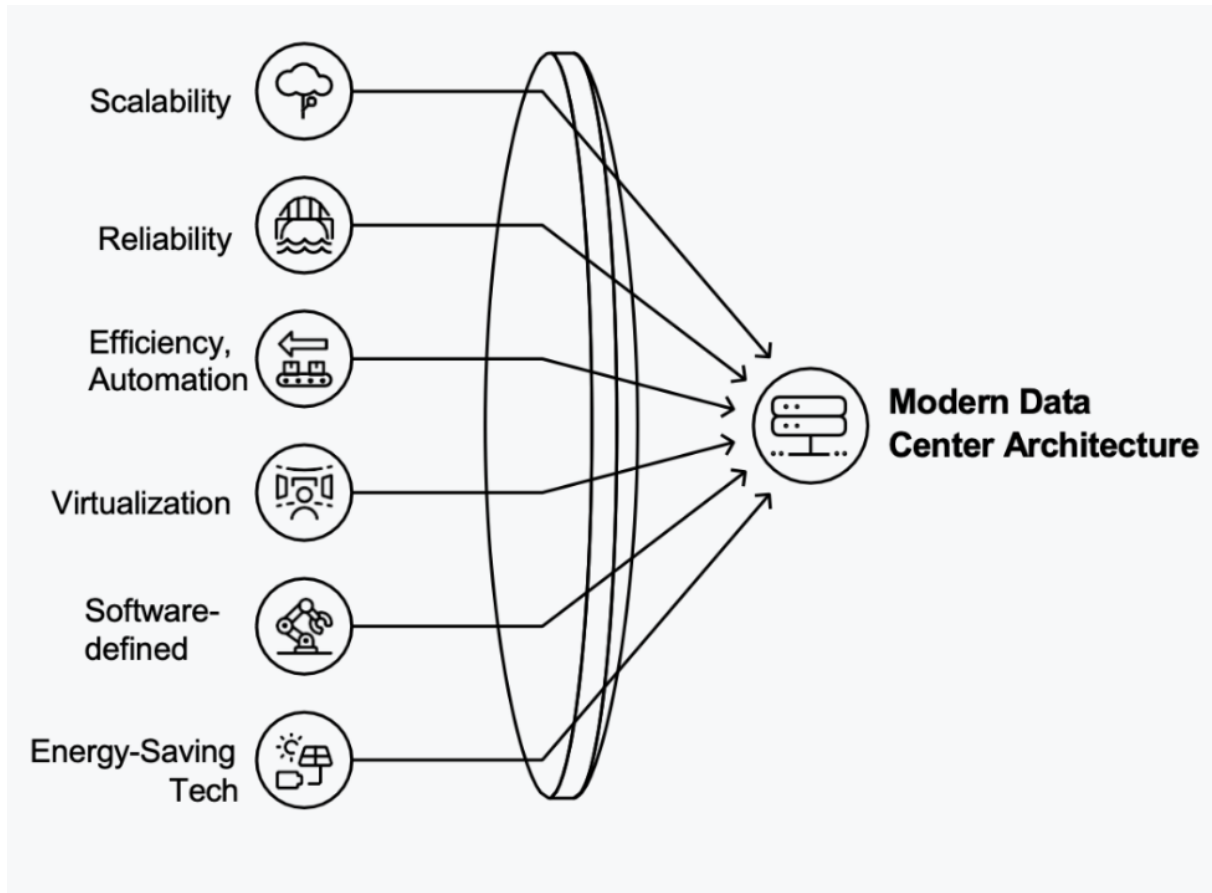


Abbildung 4. Archietekutr eines Rechenzentrum

8.1 Komponenten eines Rechenzentrum

- Netzteile (PDU)
- Kühl Systeme
- Server
- Managment Software und Automation
- Physische Sicherheits Vorkehrung
- Netzwerk Eqiument
- Speicher Systeme

8.2 Aufrecht erhaltung eine Rechenzentrum

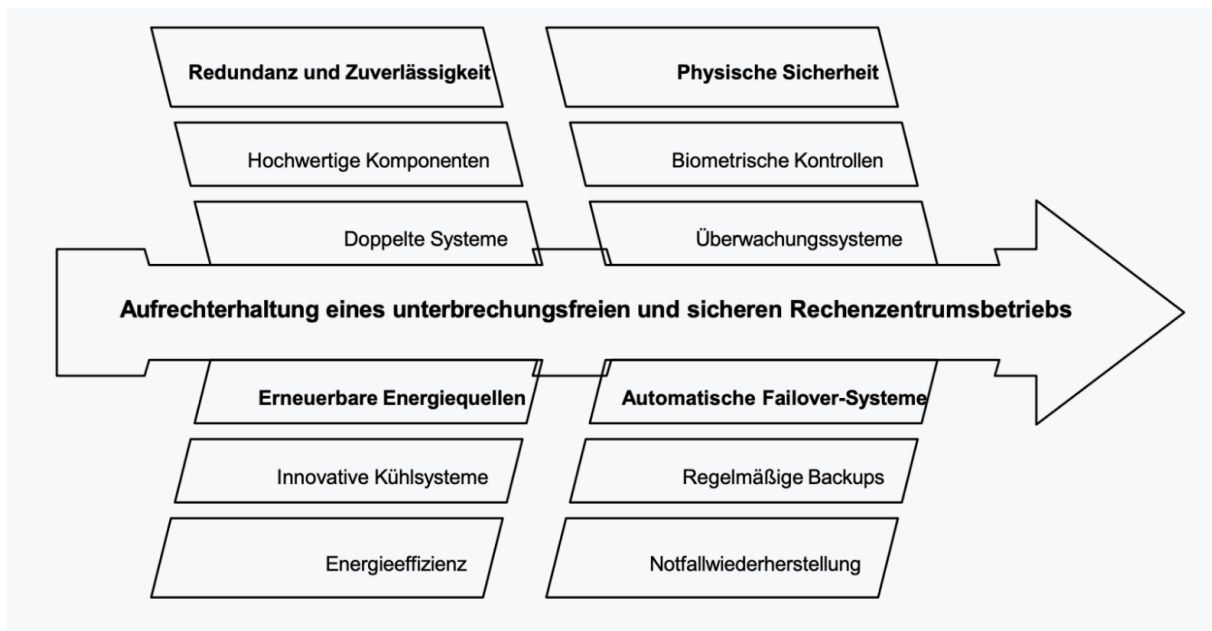


Abbildung 5. Aufrechterhaltung eines unterbrechungsfreien und sicheren Rechenzentrumsbetriebs

9 Internet Service Provider – ISP

ISP bezeichnet ein Unternehmen, das Endkunden und Organisationen den Zugang zum Internet sowie damit verbundene Dienste wie Webhosting, Domain-Registrierung und E-Mail-Hosting ermöglicht.

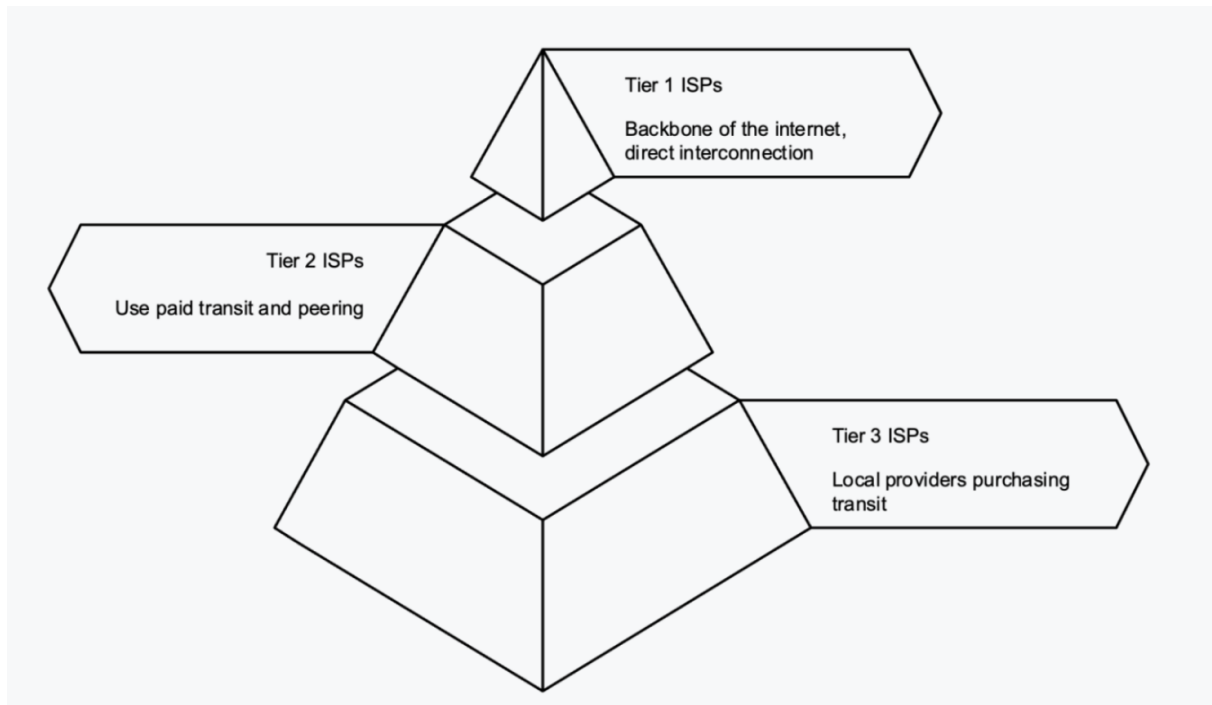


Abbildung 6. ISP Hierarchie

10 Hyperscaler

Hyperscaler sind die führenden Cloud Provider. Die Drei Großen sind AWS, Azure und Google Cloud.

Merkmal	Microsoft Azure	Amazon Web Services	Google Cloud Platform
Regionen	33 Regionen	49 Regionen	40+ Regionen
Verfügbarkeitszonen	105 Zonen	148 Zonen	121 Zonen
Besondere Daten-dienste	Azure Synapse Analytics, Azure Cosmos DB	Amazon Redshift, DynamoDB	BigQuery (serverloser Data Warehouse Dienst), Spanner
Typische Anwendungsfälle	Unternehmen mit Microsoft-Umgebung, Hybrid Cloud, Legacy-Migration	Startups, große skalierbare Workloads, breite Service-Auswahl	ML-lastige Anwendungen, Big Data, Cloud-Native-Entwicklung
Marktposition	Nr. 2 weltweit	Nr. 1 weltweit	Nr. 3 weltweit

Tabelle 4. Vergleich der Cloud-Anbieter: Azure, AWS und Google Cloud Platform



Abbildung 7. Marktanteil der Cloudprovider.

11 Docker und Kubernetes

11.1 Docker

Docker ist eine Containerisierungs-Plattform. Sie erlaubt es, Anwendungen zusammen mit allem, was sie zum Laufen brauchen, in sogenannte Container zu verpacken. Vor Container gabe es oft den Spruch "Works on my Machine" da ein entwickler andere abhängerkeiten installiert hatte als der Andere Kollege. Das Problem ist durch Docker Gelöst da man alle abhängerkeiten mitgibt.

11.1.1 Image

Ein Image ist die Blueprint also die Vorlage für einen Container. Es wird alles enthalten, was zum Ausführen eines Containers erforderlich ist. Ein Image kann in eine Docker Compose Referenziert werden. in dieser werden auch noch einstellungen vorgenommen wie envioment Variabeln.

Bespiel eine Docker Compose mit der MySQL ausgeführt wird:

```
services:
  mysql:
    image: mysql:8.0
    container_name: mysql_dev
    restart: unless-stopped

    command:
      - --default-authentication-plugin=mysql_native_password
      - --bind-address=0.0.0.0

    ports:
      - "3307:3306"

    environment:
      MYSQL_ROOT_PASSWORD: Levin-Dev-SQL

    volumes:
      - mysql_data:/var/lib/mysql
      - ./init.sql:/docker-entrypoint-initdb.d/init.sql

volumes:
  mysql_data:
```

11.1.2 Container

Ausführbare Pakete mit Anwendungscode, Laufzeit, Bibliotheken und Abhängigkeiten. Sie sind zustandslos und unveränderlich. Bei Coed Anderung müssen sie neu Erstellt werden.

11.1.3 Container Regsitrie

Eine Container Registrei ist der Ort an dem Images gespeichert werden. Es gibt Öffentli-che Registries wie besipeilweise ein Docker Hub aber auch jeder Cloud anbieter beitet

die Möglichkeit eine einzige Unternehmens Interne Container Registry zu Hosten.

11.2 Kubernetes

Kubernetes oder auch K8 wurde von Google entwickelt. Kubernetes ist eine Plattform um Container zu orchestrieren. Es verwaltet Container (nicht nur von Docker). Durch Kubernetes sind Hochverfügbarkeit, Skalierbarkeit und Disaster Recovery möglich.

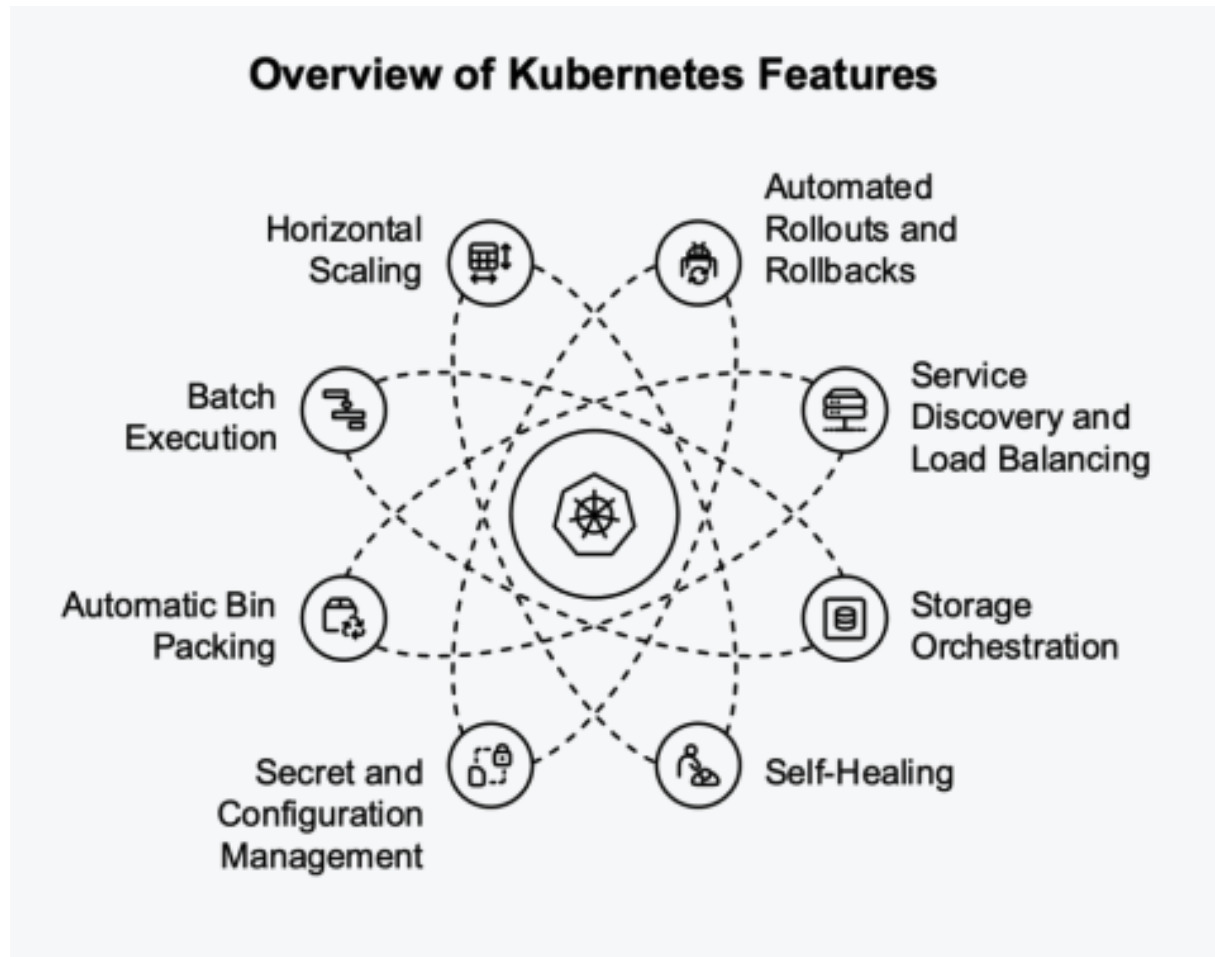


Abbildung 8. Kubernetes Features

11.2.1 Pods

Ein Pod ist die kleinste Einheit in Kubernetes. Ein Pod kann aus einem oder mehreren Containern bestehen. Alle Container in einem Pod haben geteilte Ressourcen wie Speicher, Netzwerk. Beispielsweise kann eine App in einem App Pod zusammengefasst werden.

11.2.2 Node

Ein Node ist ein Server bzw. Rechner eines Clusters.

11.2.3 Cluster

Ein Cluster ist eine Sammlung von Nodes, die gemeinsam Container und Pods ausführen.

11.2.4 Instanzen

Eine einzelne laufende Kopie einer Ressource.

11.3 Kuberets Funktion

Ein Kuberetes Cluste benötigt eine Master Node. Diese wird zu Copnmtroll Plane der die Container auf den Worker Nodes verwaltet. Wenn ein Node ausfällt hat Kuberets die Self healiing funktion und fñrt den Container auf einen anderen Node aus. Ein CLuste hat einen Virtuelles NETzwek damit die Nodes unterinander kommniziren können.

12 Cloud Architektur

12.1 Cloud Native

Cloud Native ist das erste mal 2013 angesprochen worden, von Netflix auf einen AWS reinvent talk. Es gibt keine Klare Defention für Cloud Native Architektur. Grundsätzlich soll Cloud Native aber dabei helfen Web Anwendung zu entwickeln die Skalierbar und Hochverfügbar sind. Gleichzeitig soll durch Cloud Native Agil entwickelt werden also eine Schnellst mögliche feature anpassung mit möglichst kleine Risiko. Dabei helfen Micoservice statt einer Monolithischen Architektur, Container, Dev Ops, CI/CD Pipilines. Auch Viele Lösungen wie Komponenten, Logging, Tracing sind Stadertsiert.



Abbildung 9. Kubernetes Features

12.1.1 DevOps

Kombination aus *Development* und *Operations*. Entwicklung und Betrieb arbeiten zusammen statt getrennt. Kern-Ideen: Automatisierung von Builds, Tests und Deployments, schnelle Feedbackschleifen (kurze Release-Zyklen) und gemeinsame Verantwortung für Code und Infrastruktur.

12.1.2 CI/CD – Continuous Integration / Continuous Delivery

CI: Jeder Code-Push wird automatisch gebaut und getestet, Fehler werden sofort sichtbar. **CD:** Nach erfolgreichem CI wird der Build automatisch in eine Umgebung

ausgeliefert (Staging oder Produktion).

12.1.3 Microservices

Architekturstil, bei dem eine Anwendung in viele kleine, unabhängige Dienste aufgeteilt wird. Jeder Service hat eine klar abgegrenzte Aufgabe, kommuniziert über APIs (REST oder Message Queues) und kann separat deployed und skaliert werden. Gegenteil: *Monolith*.

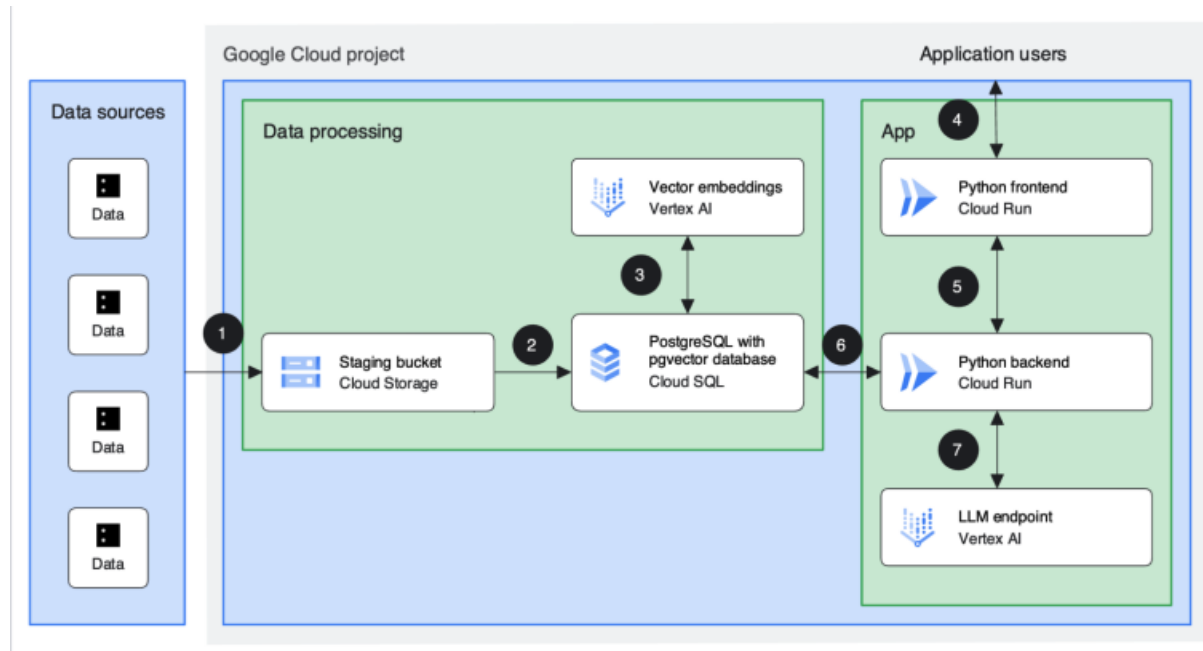


Abbildung 10. Kubernetes Features

12.2 Architektur-Grundlagen

12.2.1 Warum wichtig?

Pläne beeinflussen spätere Entscheidungen. Viele Änderungsanfragen sind vermeidbar, sonst entstehen technische Schulden. Framework-Änderungen allein lösen keine Grundprobleme, und eingeschränkte Komponentenwahl wirkt sich auf Folgeprojekte aus (z. B. Lizenzen).

12.2.2 Technische Schulden

Definition: Konsequenzen bei schlechter Umsetzung.

Arten:

- Bewusste Schulden (strategische Entscheidungen)
- Unbeabsichtigte Schulden durch mangelnde Dokumentation, fehlende Automatisierung, fehlende Coding Standards oder Missachtung von Code-Hinweisen

12.2.3 Ziele einer Lösungs-Architektur

- Skalierbarkeit, Resilienz, Sicherheit, Compliance
- Globaler Kundendienst mit gleicher Performance
- Wartbarkeit über Jahrzehnte

12.2.4 Architektur-Perspektiven

- **Geschäftssicht:** Ziele, Vision, Stakeholder, Geschäftsanforderungen, Erfolgsmetriken
- **Prozesssicht:** Abläufe von Kunde zu Server, z. B. als Sequenzdiagramm oder BPMN
- **Konzeptionelle Sicht:** Architektur ohne spezifische Cloud-Provider-Komponenten
- **Lösungssicht:** Cloud-Service-Architektur, User Flow, ER-Diagramm
- **Sicherheitssicht:** Zugangsverwaltung, Compliance-Anforderungen, Firewall

12.2.5 Architektur-Frameworks: Governance

Dienen der unternehmensweiten Steuerung, Compliance-Nachweisen und Risikokontrolle über vordefinierte Prinzipien, Gremien und Reviews. Beispiele: TOGAF, ISO/IEC/IEEE 42010:2022, Zachman-Framework.

TOGAF (The Open Group Architecture Framework): Framework zur Entwicklung und Verwaltung von Unternehmensarchitekturen (People, Processes, Technology) mit vier Domänen:

- Geschäftsarchitektur
- Informationssystemarchitektur (Anwendungs- und Datenarchitektur)
- Technologiearchitektur

12.2.6 Architektur-Frameworks: Praxisorientiert

Stellen schnelle, verständliche Diagramme für Entwickler, DevOps und Produktteams bereit. Zielen eher auf einzelne Lösungen und Applikationen ab. Beispiele: C4-Modell, 4+1-Modell, PlantUML.

C4-Modell: Unterstützt Software-Entwicklungsteams, ihre Architektur bei Planung und Durchführung zu erklären. 4 Ebenen: Software System, Container, Component, Code.

Software Architecture Diagramming Maturity Model:

- **Level 1 – Initial:** Keine Architekturdiagramme
- **Level 2 – Ad hoc:** Diagramme mit ad-hoc-Abstraktionen in einem allgemeinen Diagramm-Tool
- **Level 3 – Defined:** Diagramme mit definierten Abstraktionen und Notation, allgemeines Tool
- **Level 4 – Modelled:** Definierte Abstraktionen in einem Modellierungs-Tool, manuell erstellt
- **Level 5 – Optimising:** Modellelemente werden teamübergreifend geteilt, als querybare Datensätze genutzt und automatisch aus Sourcecode bzw. Deployment-Umgebung generiert

12.3 TOGAF – Architecture Development Method (ADM)



Abbildung 11. Kubernetes Features

Der ADM-Zyklus beschreibt, wie eine Unternehmensarchitektur schrittweise entwickelt wird. Im Zentrum stehen immer die **Requirements** (Anforderungen), auf die jede Phase kontinuierlich einzahlt.

- **Preliminary:** Framework und Prinzipien festlegen (Vorbereitung, noch kein offizieller Buchstabe)
- **A – Architecture Vision:** Ziele, Scope und Stakeholder-Erwartungen definieren
- **B – Business Architecture:** Geschäftsprozesse und -strukturen modellieren
- **C – Information System Architecture:** Anwendungs- und Datenarchitektur beschreiben
- **D – Technology Architecture:** Infrastruktur, Plattformen und Technologien festlegen
- **E – Opportunities & Solutions:** Lücken zwischen Ist- und Soll-Zustand identifizieren, Lösungsoptionen bewerten

- **F – Migration Planning:** Roadmap und Reihenfolge der Umsetzung planen
- **G – Implementation Governance:** Umsetzung begleiten und sicherstellen, dass Architekturvorgaben eingehalten werden
- **H – Architecture Change Management:** Änderungen am laufenden System kontrolliert einführen und den Zyklus bei Bedarf neu starten

Der Zyklus ist **iterativ**, d. h. nach Phase H kann der Prozess wieder bei A (oder sogar Preliminary) beginnen, falls sich Anforderungen grundlegend ändern.

12.4 C4

Das C4-Modell ist ein praxisorientiertes Framework zur Visualisierung von Software-Architektur. Der Name steht für die vier Abstraktionsebenen: **Context, Container, Component und Code**. Die Idee dahinter ist, dass verschiedene Zielgruppen unterschiedliche Detailtiefe brauchen, ein Manager braucht nur den groben Überblick, ein Entwickler hingegen die genaue Komponentenstruktur.

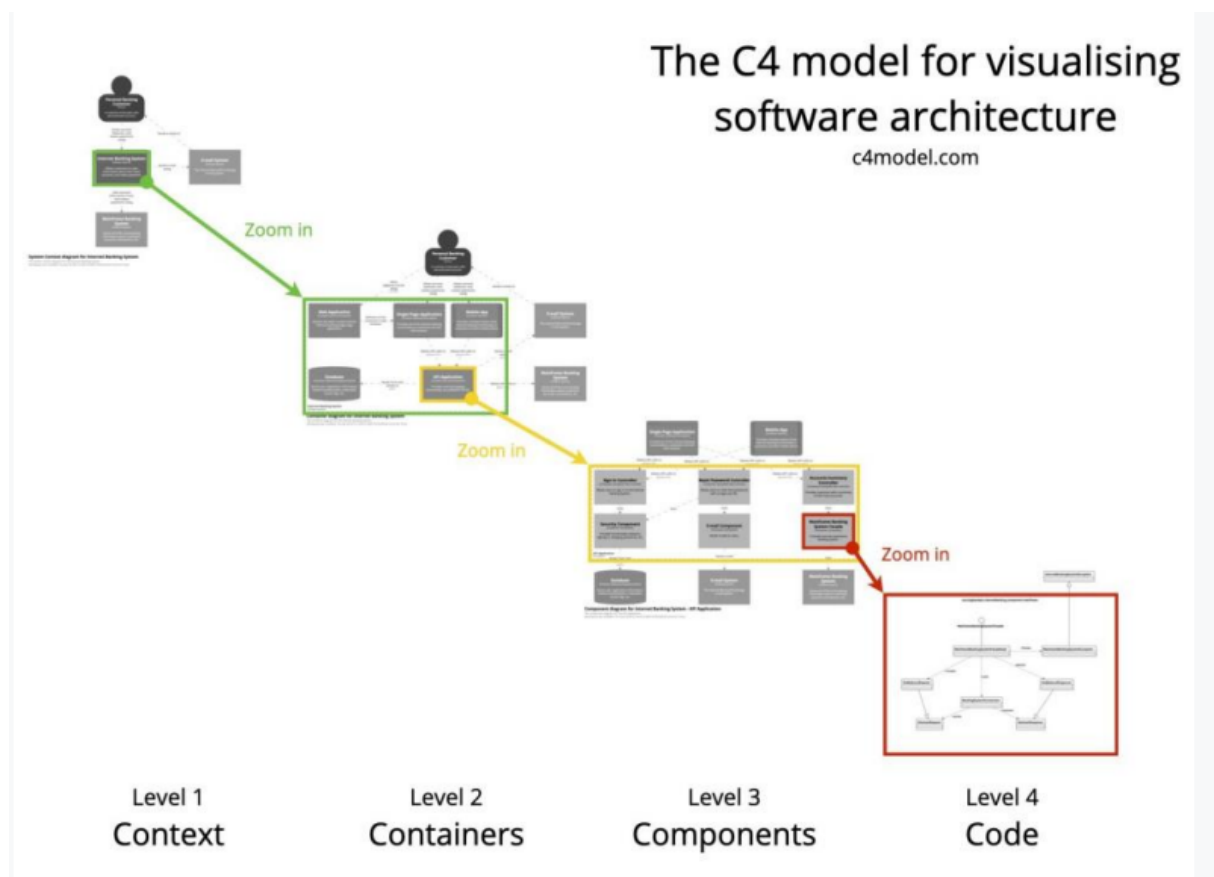


Abbildung 12. Kubernetes Features

Die vier Ebenen bauen aufeinander auf:

- **System Context:** Zeigt das System als Ganzes und seine Beziehungen zu Nutzern und externen Systemen
- **Container:** Aufschlüsselung in deploybare Einheiten wie Web-Apps, Datenbanken oder Microservices

- **Component:** Interne Struktur eines Containers, z. B. einzelne Module oder Services
- **Code:** Klassendiagramme oder ähnliches auf Code-Ebene (wird selten gezeichnet, da IDEs das übernehmen)

Ein großer Vorteil des C4-Modells ist die **Werkzeugfreiheit**. Es funktioniert mit Plant-UML, draw.io oder sogar auf einem Whiteboard, solange die vier Ebenen klar getrennt bleiben.